

Stage 2: Representative Paper and Method Selection

Kernighan–Lin Refinement for Balanced Graph Partitioning

Team Members: Name 1, Name 2, Name 3

CS240: Algorithm Design and Analysis, Spring 2026

1 Algorithmic Preliminaries

Stage 1 formulated the application as a balanced graph partitioning problem. Before selecting a specific paper, we clarify the course-level algorithmic ideas behind the methods used in this project.

First, balanced graph partitioning is a constrained optimization problem. The objective is to minimize the cut, but the partition must also remain balanced. This is different from simply finding any small cut. A very small cut may put almost every vertex on one side, which is useless for distributing work across several workers. The balance constraint is what makes the problem closer to the hard graph partitioning problems discussed in algorithms.

Second, the project compares three algorithm design styles. The greedy baseline makes locally reasonable choices while constructing a partition. Spectral bisection relaxes a discrete graph problem into a linear-algebraic problem involving the graph Laplacian. Kernighan–Lin uses local search: it starts from an existing solution and repeatedly tries to improve it by exchanging vertices. These are not unrelated tricks; they are three standard ways to handle difficult discrete optimization problems when an exact optimum is too costly.

Third, this stage uses asymptotic analysis to make the methods comparable. Instead of only asking which method gives a lower cut, we also ask how much time it takes and how the cost depends on $n = |V|$, $m = |E|$, and the number of parts k . This prepares the later implementation and experimental stages.

2 Selected Representative Method

The representative method selected for reproduction is the Kernighan–Lin graph partitioning heuristic. The original paper studies the problem of partitioning a graph into two equal-sized parts while minimizing the number or weight of edges crossing the cut. The method is a classical local-search algorithm: it starts from an initial partition, evaluates the gain of swapping vertices between the two parts, performs a sequence of beneficial swaps, and keeps the prefix of swaps that gives the largest reduction in cut value.

We select this method because it is a good match for the course project requirements. It is a classical algorithmic procedure, it has a clear formal problem statement, it admits meaningful complexity analysis, and it can be connected naturally to a modern distributed optimization application. In our project, the method is used as a refinement step for balanced k -way graph partitioning. Since the

original Kernighan–Lin method is a bisection method, we adapt it by applying pairwise refinement to pairs of current parts.

3 Formal Problem Definition

Let $G = (V, E, w)$ be an undirected weighted graph. For any partition of the vertices into two sets A and B , the cut value is

$$C(A, B) = \sum_{u \in A, v \in B, (u, v) \in E} w_{uv}.$$

The balanced bisection version asks for

$$\min_{A, B} C(A, B) \quad \text{subject to} \quad A \cup B = V, \quad A \cap B = \emptyset, \quad |A| = |B|$$

or a near-balanced version when $|V|$ is odd.

Our project generalizes the setting to k parts. The output is a mapping $p : V \rightarrow \{1, \dots, k\}$, where $p(u)$ is the worker or partition assigned to vertex u . The k -way cut objective is

$$\text{cut}(p) = \sum_{(u, v) \in E, p(u) \neq p(v)} w_{uv}.$$

The balance ratio is measured as

$$\rho(p) = \frac{\max_i |\{v \in V : p(v) = i\}|}{|V|/k}.$$

A perfectly balanced partition has $\rho(p) = 1$, while values slightly above one indicate mild imbalance.

4 Core Algorithm Description

For graph bisection, Kernighan–Lin assigns each vertex an external cost and an internal cost. For a vertex $a \in A$, the internal cost $I(a)$ is the total weight of edges from a to other vertices in A , while the external cost $E(a)$ is the total weight of edges from a to vertices in B . The difference

$$D(a) = E(a) - I(a)$$

measures how much a prefers the opposite side. If $a \in A$ and $b \in B$ are swapped, the gain is

$$g(a, b) = D(a) + D(b) - 2w_{ab},$$

where $w_{ab} = 0$ if there is no edge between them. A positive gain means the swap reduces the cut.

A pass of the algorithm proceeds as follows:

1. Start from a balanced partition (A, B) .
2. Mark all vertices as unlocked.
3. Repeatedly choose the unlocked pair (a, b) with largest gain.
4. Lock the chosen vertices and record the swap gain.

5. After all possible pairs are selected, find the prefix of swaps with maximum cumulative gain.
6. Apply that prefix only if the maximum cumulative gain is positive.

The process repeats for several passes until no further improvement is found. The important idea is that the algorithm may temporarily select a swap with negative immediate gain because a later sequence of swaps can still produce a positive cumulative improvement.

5 Adaptation to k -Way Partitioning

The original method operates on two sets. Our implementation uses the same local-search principle inside a k -way framework. First, we compute an initial k -way partition using recursive spectral bisection. Then, for every pair of parts (i, j) , we extract the subgraph induced by vertices currently assigned to those two parts. We run a pairwise Kernighan–Lin bisection on that induced subgraph using the current two parts as the initial bisection. If the resulting local update does not increase the global cut, we accept it.

This adaptation is practical for the course project because it keeps the core Kernighan–Lin refinement idea while making the method applicable to the project’s k -worker distributed optimization setting. It also gives a clean comparison against the two baselines:

- a greedy balanced partitioning baseline, which is simple and fast;
- recursive spectral bisection, which provides a stronger linear-algebra-based initial partition.

6 Complexity Analysis

Let $n = |V|$, $m = |E|$, and k be the number of parts.

Greedy baseline. The greedy method assigns vertices one by one. For each vertex, it evaluates the feasible parts and computes a local incremental cut cost from already assigned neighbors. A straightforward implementation has cost roughly $O(mk)$ or $O(nk + \sum_v \deg(v)k)$ depending on how costs are maintained. In our small benchmark implementation, this is fast and serves as a lower-runtime baseline.

Spectral bisection. Recursive spectral bisection computes a graph Laplacian and an eigenvector at each split. With dense eigen-decomposition, a split on s vertices costs $O(s^3)$. Recursive bisection therefore has a cost dominated by the largest splits. This is acceptable for course-scale experiments but would need sparse eigensolvers for larger graphs.

Kernighan–Lin refinement. For a two-way subproblem with s vertices, a naive pass can evaluate many pairs and may cost about $O(s^2)$ or more depending on data structures. The overall k -way refinement considers $\binom{k}{2}$ pairs of parts for several passes. If T is the number of global refinement passes and each pair subproblem has size about $2n/k$, a rough estimate is

$$O\left(T \binom{k}{2} \left(\frac{2n}{k}\right)^2\right) = O(Tn^2)$$

for the pairwise local-search component, ignoring graph sparsity and library implementation details. In practice, runtime depends strongly on graph density, the number of accepted refinements, and the stopping condition.

7 Planned Reproduction Targets

The project does not attempt to reproduce every experimental table from the original Kernighan–Lin paper. Instead, it reproduces the central algorithmic behavior expected from the method:

1. starting from an initial partition;
2. applying Kernighan–Lin-style local refinement;
3. reducing the cut value compared with the initial partition or a simpler baseline;
4. maintaining balanced part sizes;
5. measuring the runtime cost of refinement.

The expected empirical results are:

- Kernighan–Lin refinement should often reduce cut weight relative to greedy partitioning.
- It should improve or match spectral partitioning on many instances, because spectral partitioning supplies a strong initial solution and the local search can repair local mistakes.
- It should require more runtime than the greedy baseline because it performs additional refinement passes.
- Lower cut weight should correspond to fewer cross-worker messages in the distributed consensus simulation.

8 Evaluation Plan

The method will be evaluated on grid graphs, Erdos–Renyi random graphs, random geometric graphs, and the Zachary karate club graph. These instances cover structured graphs, random graphs, spatial graphs, and a small real social network. The metrics are cut weight, normalized cut, balance ratio, runtime, final consensus error, and cross-partition message count.

This evaluation plan directly connects the representative method to the project goal: showing how a classical graph partitioning heuristic can reduce communication cost in a modern distributed optimization setting.

References

- [1] B. W. Kernighan and S. Lin. An efficient heuristic procedure for partitioning graphs. *Bell System Technical Journal*, 1970.
- [2] G. Karypis and V. Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 1998.